

Ejercicios 1

Víctor Manuel Peiró Martínez

FIIS 3A

Ampliación de Bases de Datos

1. **Insertar un solo documento:** inserte un nuevo documento de usuario en una colección de users.

```
db.users.insertOne({_id:11,firstName: 'Victor', lastName:'Peiro', email:
'victor.peiro@example.com', age:20,username:'victorp',lastLogin:'2025-09-
22'})
```

```
exercises> db.users.insertOne({_id:11,firstName: 'Victor', lastName:'Peiro', email: 'victor.peiro@example.com', age:20,username:'victorp',lastLogin:'2025-09-22'})
{ acknowledged: true, insertedId: 11 }
```

2. **Insertar varios documentos:** inserte varios documentos de productos en una colección de products.

```
exercises> db.products.insertMany([({_id:11, name:'T-Shirt',description:'Sport T-Shirt',price: 45,category: 'Sportswear'},({_id:12,name:'Sofa',description:'Comfortable Sofa',price:
500,category: 'Furniture'})])
{ acknowledged: true, insertedIds: { '0': 11, '1': 12 } }
```

3. **Buscar un documento por ID:** Recupere un documento de usuario por su _id de la colección de users

```
exercises> db.users.find({_id:5})
[
  {
    _id: 5,
    firstName: 'Charlie',
    lastName: 'Davis',
    email: 'charlie.davis@example.com',
    age: 31,
    username: 'charlied',
    lastLogin: '2024-07-22'
  }
]
```

4. **Actualizar un documento:** actualice la dirección de correo electrónico de un usuario en la colección de users.

```
exercises> db.users.updateOne({email: 'charlie.davis@example.com'},{$set:{email:'charliedavis@example.com'}})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
exercises> db.users.find({_id:5})
[
  {
    _id: 5,
    firstName: 'Charlie',
    lastName: 'Davis',
    email: 'charliedavis@example.com',
    age: 31,
    username: 'charlied',
    lastLogin: '2024-07-22'
  }
]
```

5. Eliminar un documento: elimina un producto de la colección de products por su _id.

```
exercises> db.products.deleteOne({_id:12})  
{ acknowledged: true, deletedCount: 1 }
```

6. Buscar documentos con una condición: recupere todos los usuarios mayores de 30 años de la colección de users

```
exercises> db.users.find({age:{$gt : 30}})
[
  {
    _id: 8,
    firstName: 'Frank',
    lastName: 'Moore',
    email: 'frank.moore@example.com',
    age: 50,
    username: 'frankm',
    lastLogin: '2023-11-30'
  },
  {
    _id: 5,
    firstName: 'Charlie',
    lastName: 'Davis',
    email: 'charliedavis@example.com',
    age: 31,
    username: 'charlied',
    lastLogin: '2024-07-22'
  },
  {
    _id: 6,
    firstName: 'David',
    lastName: 'Wilson',
    email: 'david.wilson@example.com',
    age: 38,
    username: 'davidw',
    lastLogin: '2024-01-15'
  },
  {
    _id: 9,
    firstName: 'Grace',
    lastName: 'Taylor',
    email: 'grace.taylor@example.com',
    age: 42,
    username: 'gracet',
    lastLogin: '2024-02-10'
  },
  {
    _id: 3,
    firstName: 'Alice',
    lastName: 'Johnson',
    email: 'alice.johnson@example.com',
    age: 45,
    username: 'alicej',
    lastLogin: '2023-12-20'
  },
  {
    _id: 10,
    firstName: 'Hank',
    lastName: 'Anderson',
    email: 'hank.anderson@example.com',
    age: 36,
    username: 'hanka',
    lastLogin: '2024-03-25'
  },
  {
    _id: 2,
    firstName: 'Jane',
    lastName: 'Smith',
    email: 'jane.smith@example.com',
    age: 34,
    username: 'janesmith',
    lastLogin: '2024-08-15'
  }
]
```

7. Actualizar varios documentos: Aumente el precio de todos los productos de la colección de products en un 10%

```
exercises> db.products.updateMany({}, {$mul: {price:1.1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 11,
  modifiedCount: 11,
  upsertedCount: 0
}
```

8. Eliminar varios documentos: elimine todos los usuarios que no hayan iniciado sesión durante más de seis meses de la colección de users.

```
exercises> db.users.deleteMany({lastLogin: {$lte : '2024-03-22'}})
{ acknowledged: true, deletedCount: 4 }
```

9. Operación Upsert: Inserte un nuevo usuario si no existe, o actualice la información del usuario existente

```
exercises> db.users.updateOne({name:"Jane"},{$set : {name:"Janet", lastName:"Smith"}},{upsert:true})
{
  acknowledged: true,
  insertedId: ObjectId('68d67b3d982b6f61fb247938'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
exercises> db.users.find({name:"Jane"})
exercises> |
```

10. Expresión regular: Encuentre places que tengan la palabra "of" en sus nombres usando una expresión regular

```
exercises> db.places.find({name: {$regex: "\\bof\\b"}})
[
  {
    _id: 4,
    name: 'Statue of Liberty',
    location: { type: 'Point', coordinates: [ -74.0445, 40.6892 ] },
    rating: 4.6
  },
  {
    _id: 6,
    name: 'Great Wall of China',
    location: { type: 'Point', coordinates: [ 116.5704, 40.4319 ] },
    rating: 4.8
  }
]
exercises> |
```

11. Índice compuesto: cree un índice compuesto en firstName y lastName en la colección de places

```
exercises> db.users.createIndex({firstName: 1, lastName: 1})
firstName_1_lastName_1
```

12. Índice de texto: cree un índice de texto en el campo de description de la colección de products para la búsqueda de texto completo

```
exercises> db.products.createIndex({description:"text"})
description_text
```

13. Índice único: asegúrese de que el campo de nombre de usuario de la colección de users sea único.

```
exercises> db.users.createIndex({username : "text"},{unique: true})
username_text
exercises> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { firstName: 1, lastName: 1 },
    name: 'firstName_1_lastName_1'
  },
  {
    v: 2,
    key: { _fts: 'text', _ftsx: 1 },
    name: 'username_text',
    unique: true,
    weights: { username: 1 },
    default_language: 'english',
    language_override: 'language',
    textIndexVersion: 3
  }
]
```

14. Buscar texto: Usando el índice de texto, busque los products con la palabra "laptop" en su descripción.

```
exercises> db.products.find({$text: {$search : "Laptop"}})
[
  {
    _id: 1,
    name: 'Laptop',
    description: 'A high-performance laptop',
    price: 1320,
    category: 'Electronics'
  }
]
```

15. Índice geoespacial: cree un índice 2dsphere en el campo de location de la colección places

```
exercises> db.places.createIndex({location:"2dsphere"})
location_2dsphere
exercises> db.places.getIndex
db.places.getIndexes    db.places.getIndexSpecs  db.places.getIndexKeys

exercises> db.places.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { location: '2dsphere' },
    name: 'location_2dsphere',
    '2dsphereIndexVersion': 3
  }
]
```

16. Recupera todos los productos con precios superiores a 1000 y con la palabra "laptop" en la descripción

```
exercises> db.products.find({$and : [{ $text: { $search: "laptop" } }, { price: { $gt: 1000 } } ] })
[
  {
    _id: 1,
    name: 'Laptop',
    description: 'A high-performance laptop',
    price: 1320,
    category: 'Electronics'
  }
]
```

17. Buscar documentos cerca de un punto: recupere todos los places dentro de los 5 kilómetros de un punto determinado

```
exercises> db.places.find({location: {$near: {$geometry: {type: "Point", coordinates: [-74.0, 40.71]}, $maxDistance: 5000}}})
[
  {
    _id: 4,
    name: 'Statue of Liberty',
    location: { type: 'Point', coordinates: [ -74.0445, 40.6892 ] },
    rating: 4.6
  }
]
```

18. Clasificación geoespacial: ordena los places por su distancia a un punto determinado.

```
exercises> db.places.find({location: {$near: {$geometry: {type: "Point", coordinates: [0,0]}, $minDistance: 0.0}}}).sort()
[
  {
    _id: 5,
    name: 'Colosseum',
    location: { type: 'Point', coordinates: [ 12.4922, 41.8902 ] },
    rating: 4.7
  },
  {
    _id: 10,
    name: 'Christ the Redeemer',
    location: { type: 'Point', coordinates: [ -43.2105, -22.9519 ] },
    rating: 4.7
  },
  {
    _id: 2,
    name: 'Eiffel Tower',
    location: { type: 'Point', coordinates: [ 2.2945, 48.8584 ] },
    rating: 4.8
  },
  {
    _id: 3,
    name: 'Louvre Museum',
    location: { type: 'Point', coordinates: [ 2.3364, 48.8606 ] },
    rating: 4.9
  },
  {
    _id: 9,
    name: 'Machu Picchu',
    location: { type: 'Point', coordinates: [ -72.545, -13.1631 ] },
    rating: 4.8
  },
  {
    _id: 1,
    name: 'Central Park',
    location: { type: 'Point', coordinates: [ -73.9654, 40.7829 ] },
    rating: 4.7
  },
  {
    _id: 4,
    name: 'Statue of Liberty',
    location: { type: 'Point', coordinates: [ -74.0445, 40.6892 ] },
    rating: 4.6
  },
  {
    _id: 8,
    name: 'Taj Mahal',
    location: { type: 'Point', coordinates: [ 78.0421, 27.1751 ] },
    rating: 4.9
  },
  {
    _id: 6,
    name: 'Great Wall of China',
    location: { type: 'Point', coordinates: [ 116.5704, 40.4319 ] },
    rating: 4.8
  },
  {
    _id: 7,
    name: 'Sydney Opera House',
    location: { type: 'Point', coordinates: [ 151.2153, -33.8568 ] },
    rating: 4.7
  }
]
```

19. Buscar documentos usando \$or : Busque places con la palabra "the" o "of" en sus nombres.

```
exercises> db.places.find({$or: [{name:{$regex:"\\bthe\\b"}},{name:{$regex:"\\bof\\b"}}]})
[
  {
    _id: 4,
    name: 'Statue of Liberty',
    location: { type: 'Point', coordinates: [ -74.0445, 40.6892 ] },
    rating: 4.6
  },
  {
    _id: 6,
    name: 'Great Wall of China',
    location: { type: 'Point', coordinates: [ 116.5704, 40.4319 ] },
    rating: 4.8
  },
  {
    _id: 10,
    name: 'Christ the Redeemer',
    location: { type: 'Point', coordinates: [ -43.2105, -22.9519 ] },
    rating: 4.7
  }
]
exercises> |
```

20. Buscar documentos mediante \$in : Buscar products de tipo "Electronics" y "Furniture"

```
exercises> db.products.find({category:{$in: ['Electronics','Furniture']}})
[
  {
    _id: 1,
    name: 'Laptop',
    description: 'A high-performance laptop',
    price: 1320,
    category: 'Electronics'
  },
  {
    _id: 2,
    name: 'Smartphone',
    description: 'A latest model smartphone',
    price: 880.0000000000001,
    category: 'Electronics'
  },
  {
    _id: 3,
    name: 'Headphones',
    description: 'Noise-cancelling headphones',
    price: 220.0000000000003,
    category: 'Electronics'
  },
  {
    _id: 6,
    name: 'Desk Chair',
    description: 'Ergonomic desk chair',
    price: 275,
    category: 'Furniture'
  },
  {
    _id: 7,
    name: 'Bookshelf',
    description: 'Wooden bookshelf',
    price: 330,
    category: 'Furniture'
  }
]
```


21. Buscar documentos por valor de matriz: busque places con ambas coordenadas menores que 0

```
exercises> db.places.find({$and: [{"location.coordinates.0":{$lt:0}}, {"location.coordinates.1":{$lt:0}}]})
[
  {
    _id: 9,
    name: 'Machu Picchu',
    location: { type: 'Point', coordinates: [ -72.545, -13.1631 ] },
    rating: 4.8
  },
  {
    _id: 10,
    name: 'Christ the Redeemer',
    location: { type: 'Point', coordinates: [ -43.2105, -22.9519 ] },
    rating: 4.7
  }
]
```